

INVESTIGACIÓN ACADÉMICA

FUNDAMENTOS DE LAS APLICACIONES WEB

*Una revisión completa sobre arquitectura, tecnologías,
protocolos, seguridad y evolución histórica de la Web*

1. Introducción

Las aplicaciones web representan uno de los pilares tecnológicos más relevantes de la era digital contemporánea. Desde correos electrónicos hasta plataformas de comercio electrónico, redes sociales, sistemas bancarios en línea y herramientas colaborativas, casi toda actividad humana mediada por la tecnología ocurre a través de software que se ejecuta en navegadores web. Comprender los fundamentos sobre los cuales se construyen estas aplicaciones es esencial tanto para el desarrollo profesional del ingeniero de sistemas como para la comprensión crítica del ecosistema digital actual.

La World Wide Web (WWW), concebida por Sir Tim Berners-Lee en 1989 mientras trabajaba en el CERN, fue originalmente diseñada como un sistema de gestión de información basado en hipertexto para facilitar el intercambio de documentos científicos entre investigadores [1] [2]. Sin embargo, en pocas décadas evolucionó hasta convertirse en la plataforma más grande y compleja de comunicación, comercio y producción de conocimiento que la humanidad haya construido jamás.

Esta investigación examina en profundidad los fundamentos técnicos, históricos y conceptuales de las aplicaciones web, abarcando: la arquitectura cliente-servidor, los protocolos de comunicación (HTTP/HTTPS), los lenguajes base (HTML, CSS, JavaScript), la distinción entre frontend y backend, las bases de datos, la seguridad web y las tendencias modernas como los frameworks, las APIs REST y las arquitecturas de microservicios.

Objetivo de la investigación:

Proporcionar una comprensión profunda, estructurada y respaldada por fuentes técnicas y académicas verificadas sobre los fundamentos de las aplicaciones web, cubriendo desde sus orígenes históricos hasta las tecnologías y paradigmas actuales.

2. Antecedentes Históricos y Evolución de la Web

2.1. Los Orígenes de Internet y la World Wide Web

Para comprender las aplicaciones web es imprescindible distinguir entre dos conceptos que frecuentemente se confunden: Internet y la World Wide Web. Internet es la infraestructura global de redes interconectadas que permite la comunicación entre millones de dispositivos usando el protocolo TCP/IP, y tiene sus raíces en ARPANET, creada por el Departamento de Defensa de los Estados Unidos en 1969 [3]. La WWW, en cambio, es un servicio que corre sobre Internet: un sistema de documentos de hipertexto accesibles mediante navegadores, propuesto por Tim Berners-Lee en 1989 y puesto en funcionamiento el 6 de agosto de 1991 [2].

Berners-Lee definió los tres pilares fundamentales de la Web: el lenguaje HTML para estructurar documentos, el protocolo HTTP para transferirlos, y el sistema URL para identificarlos de forma única en la red [1]. Estos tres elementos, con sus respectivas evoluciones, siguen siendo la columna vertebral de la Web actual más de tres décadas después de su creación.

2.2. Web 1.0: La Web Estática (1991–2003)

La primera generación de la Web, conocida retrospectivamente como Web 1.0, se caracterizó por ser esencialmente unidireccional y de solo lectura. Los sitios web eran páginas estáticas escritas en HTML puro, alojadas en servidores que respondían peticiones pero sin capacidad de interacción dinámica con el usuario. El contenido era producido únicamente por los propietarios de los sitios, y los visitantes eran consumidores pasivos de información [4].

En 1993 el NCSA lanzó Mosaic, el primer navegador en mostrar imágenes junto al texto, lo que impulsó masivamente la adopción de la Web. Ese mismo año se introdujo la interfaz CGI (Common Gateway Interface), el primer mecanismo para generar páginas web dinámicamente ejecutando un programa en el servidor por cada petición [1]. Esta fue la semilla de la programación del lado del servidor.

2.3. Web 2.0: La Web Participativa (2004–2015)

El término 'Web 2.0' fue popularizado por Tim O'Reilly y Dale Dougherty en 2004. No designó una actualización técnica del protocolo, sino un cambio de paradigma caracterizado por la participación activa del usuario, la colaboración en línea y el contenido generado por la comunidad [4].

Tecnologías clave que lo habilitaron incluyeron AJAX, XML, JSON y la maduración de JavaScript como lenguaje del lado del cliente.

Un hito fundamental fue el lanzamiento en 2004 de Gmail y Google Maps, que demostraron que era posible construir experiencias de usuario comparables a las aplicaciones de escritorio nativas utilizando exclusivamente tecnologías web. Esto transformó la percepción de lo que era posible lograr dentro de un navegador.

2.4. Web 3.0 y Más Allá

La visión más consolidada académicamente de la Web 3.0 es la de la Web Semántica, propuesta por Berners-Lee, Hendler y Lassila en 2001 en Scientific American. Propone añadir metadatos estructurados a los documentos web para que las máquinas puedan interpretarlos con mayor precisión y razonar sobre su contenido [5]. En la actualidad, Web 3.0 también se asocia con blockchain, contratos inteligentes y aplicaciones descentralizadas (DApps), mientras que el concepto de Web 4.0 apunta a una web cognitiva que integra Inteligencia Artificial, Big Data e IoT.

Generación	Período aprox.	Característica principal	Tecnologías clave
Web 1.0	1991–2003	Solo lectura, páginas estáticas	HTML, HTTP, FTP, CGI
Web 2.0	2004–2015	Lectura-escritura, participación	AJAX, JavaScript, PHP, RSS
Web 3.0	2015–presente	Web semántica, descentralización	Blockchain, IA, RDF, JSON-LD
Web 4.0	En desarrollo	Web cognitiva e inteligente	IoT, Big Data, ML, AR/VR

3. Arquitectura Cliente-Servidor

3.1. Concepto y Fundamentos

La arquitectura cliente-servidor es el modelo de diseño de software que subyace a la totalidad de las aplicaciones web. En este modelo, el sistema se divide en dos componentes con roles bien definidos: el cliente, que solicita recursos o servicios; y el servidor, que los provee [9]. Esta separación de responsabilidades permite escalar, mantener y asegurar las aplicaciones de manera más eficiente que en los modelos monolíticos.

El principio fundamental es que el cliente inicia la comunicación solicitando un servicio, y el servidor responde proveyendo el resultado. Este principio, aunque simple en su formulación, da lugar a sistemas de enorme complejidad como las plataformas web globales de hoy [6].

3.2. El Cliente Web: El Navegador

En el contexto de las aplicaciones web, el cliente es el navegador web (agente de usuario). Es el software encargado de interpretar y renderizar los documentos HTML, aplicar hojas de estilo CSS, ejecutar JavaScript y gestionar la comunicación con los servidores mediante HTTP. El navegador envía una petición HTTP al servidor para obtener un documento HTML, lo procesa y realiza peticiones adicionales para scripts, hojas de estilo e imágenes, construye el árbol DOM y el árbol CSSOM para renderizar la página visualmente.

3.3. El Servidor Web

El servidor web recibe las peticiones HTTP de los clientes, las procesa y devuelve las respuestas correspondientes. Puede entregar recursos estáticos directamente desde el sistema de archivos, o delegar la generación de contenido dinámico a una aplicación del lado del servidor. En entornos de alta demanda, los servidores web trabajan con balanceadores de carga que distribuyen las peticiones entre múltiples instancias para garantizar disponibilidad.

3.4. Arquitectura en Capas (N-Tier)

Las aplicaciones web modernas utilizan comúnmente una arquitectura de tres capas que separa la lógica de presentación, la lógica de negocio y el acceso a datos [7]:

- Capa de Presentación (Frontend): Interfaz de usuario que se ejecuta en el navegador. Se construye con HTML, CSS y JavaScript.

- Capa de Lógica de Negocio (Backend): Servidor de aplicaciones que implementa las reglas de negocio, coordina el acceso a datos y genera las respuestas.
- Capa de Datos (Persistencia): Sistema de gestión de bases de datos (SGBD) que almacena y recupera la información de manera persistente.

4. Protocolos de Comunicación Web

4.1. El Protocolo HTTP

HTTP (HyperText Transfer Protocol) es el fundamento de toda comunicación en la WWW. Fue diseñado a principios de la década de 1990 por Tim Berners-Lee en el CERN. Desde el punto de vista técnico, es un protocolo de la capa de aplicación (Capa 7 del modelo OSI) que se transmite sobre TCP/IP. Es un protocolo sin estado (stateless): el servidor no conserva información entre peticiones sucesivas del mismo cliente, lo que garantiza escalabilidad pero requiere mecanismos adicionales como cookies y sesiones para implementar experiencias con estado.

4.1.1. Versiones de HTTP

- HTTP/0.9 (1991): Solo admitía el método GET y retornaba únicamente HTML sin cabeceras.
- HTTP/1.0 (RFC 1945, 1996): Añadió cabeceras, códigos de estado, tipos MIME y los métodos POST y HEAD.
- HTTP/1.1 (RFC 2616, 1999 / RFC 7230, 2014): Introdujo conexiones persistentes, pipeline de peticiones y host virtual. Fue el estándar dominante durante casi dos décadas [8] [9].
- HTTP/2 (RFC 7540, 2015): Introdujo multiplexación de peticiones sobre una sola conexión TCP, compresión de cabeceras (HPACK) y comunicación binaria, mejorando significativamente el rendimiento [10].
- HTTP/3 (RFC 9114, 2022): Reemplaza TCP por QUIC (sobre UDP), eliminando el bloqueo de cabecera de línea también en la capa de transporte [11].

4.1.2. Métodos HTTP (RFC 7231)

Método	Descripción	Uso típico
GET	Solicita la representación de un recurso. Solo recupera datos.	Cargar páginas, obtener datos de API
POST	Envía datos al servidor para crear un recurso o procesar información.	Formularios, crear registros en API
PUT	Reemplaza completamente un recurso existente.	Actualizar un registro completo
PATCH	Aplica modificaciones parciales a un recurso.	Actualizar campos específicos
DELETE	Elimina el recurso especificado.	Eliminar registros

Método	Descripción	Uso típico
HEAD	Como GET pero sin cuerpo de respuesta.	Verificar existencia de recurso
OPTIONS	Describe las opciones de comunicación disponibles.	Mecanismo CORS preflight

4.1.3. Códigos de Estado HTTP

Código	Categoría	Significado
200 OK	Éxito (2xx)	Petición procesada correctamente
201 Created	Éxito (2xx)	Recurso creado (común en APIs REST)
301 Moved Permanently	Redirección (3xx)	Recurso movido permanentemente
400 Bad Request	Error del cliente (4xx)	Errores sintácticos en la petición
401 Unauthorized	Error del cliente (4xx)	Se requiere autenticación
403 Forbidden	Error del cliente (4xx)	Acceso denegado
404 Not Found	Error del cliente (4xx)	Recurso no encontrado
500 Internal Server Error	Error del servidor (5xx)	Error interno del servidor

4.2. HTTPS y TLS

HTTPS (HTTP Secure) añade una capa de cifrado mediante TLS (Transport Layer Security). TLS 1.3, estandarizado en RFC 8446 (2018), garantiza tres propiedades: (1) confidencialidad, cifrando la comunicación para que no pueda ser leída por terceros; (2) integridad, garantizando que los datos no han sido alterados en tránsito; y (3) autenticación, verificando mediante certificados digitales la identidad del servidor [12]. Los navegadores modernos marcan los sitios HTTP como 'no seguros', y el uso de HTTPS es hoy un estándar obligatorio de la industria.

4.3. DNS: Sistema de Nombres de Dominio

El DNS (RFC 1034, 1987) es el servicio responsable de traducir nombres de dominio legibles (www.ejemplo.com) en direcciones IP numéricas que las máquinas utilizan para comunicarse. Opera como un sistema jerárquico y distribuido de bases de datos [13]. Cuando un usuario escribe

una URL, se consulta primero la caché local, luego el servidor DNS del ISP o uno público (Google 8.8.8.8, Cloudflare 1.1.1.1), escalando hasta el servidor autoritativo del dominio. Todo el proceso suele completarse en milisegundos.

5. Tecnologías del Lado del Cliente (Frontend)

5.1. HTML5

HTML (HyperText Markup Language) es el lenguaje de marcado estándar para crear páginas web. Define la estructura semántica del contenido mediante etiquetas. La versión HTML5, cuya especificación fue presentada el 28 de octubre de 2014 por el W3C [14], incorporó: soporte nativo para audio y vídeo sin plugins externos, el elemento Canvas para gráficos mediante JavaScript, APIs de geolocalización y almacenamiento local, Web Workers, Web Sockets y un conjunto ampliado de elementos semánticos (<header>, <footer>, <article>, <section>, <nav>, <aside>). La semántica facilita la accesibilidad, mejora el SEO y facilita el mantenimiento del código [15].

5.2. CSS3

CSS (Cascading Style Sheets) describe la presentación visual de un documento HTML, separando presentación de contenido [16]. CSS3 está dividido en módulos independientes que se estandarizan por separado: Flexbox (diseños de una dimensión), CSS Grid (diseños de dos dimensiones), Transiciones y Animaciones, y Variables CSS (Custom Properties). El diseño responsive, habilitado por las media queries, permite que las aplicaciones web adapten su diseño a diferentes tamaños de pantalla, desde teléfonos móviles hasta monitores de alta resolución.

5.3. JavaScript y ECMAScript

JavaScript es el único lenguaje de programación que los navegadores ejecutan de manera nativa. Creado en 1995 por Brendan Eich en Netscape, su estandarización la gestiona ECMA International bajo el estándar ECMAScript (ECMA-262), publicándose versiones anuales desde ES2015 [17]. Es un lenguaje interpretado, de tipado dinámico, orientado a objetos basado en prototipos y con soporte para programación funcional [18].

En la actualidad, JavaScript también se ejecuta en el servidor gracias a Node.js (Ryan Dahl, 2009). Frameworks como React (Meta, 2013), Angular (Google, 2016), Vue.js (Evan You, 2014) y Next.js han transformado radicalmente la construcción de interfaces web, introduciendo componentes reutilizables, estado reactivo, renderización del lado del servidor (SSR) y generación de sitios estáticos (SSG) [18].

5.4. El DOM

El Document Object Model (DOM) es la interfaz estándar que representa la estructura de un documento HTML como un árbol de objetos en memoria. JavaScript puede leer, modificar, añadir

o eliminar nodos para actualizar dinámicamente la interfaz sin recargar la página completa. Esta capacidad fue uno de los factores clave que habilitó las aplicaciones web de página única (SPA).

6. Tecnologías del Lado del Servidor (Backend)

6.1. Rol del Backend

El backend es la parte de la aplicación web que se ejecuta en el servidor y es invisible para el usuario final. Su responsabilidad es implementar la lógica de negocio: autenticar usuarios, validar datos, procesar peticiones, interactuar con bases de datos y generar respuestas para el cliente. También se ocupa de la seguridad, la escalabilidad y la integridad de los datos [7].

6.2. Lenguajes y Frameworks del Servidor

- PHP (1994): Impulsa más del 75% de los sitios web con lenguaje del lado del servidor conocido, incluyendo WordPress. Frameworks: Laravel, Symfony.
- Python: Ampliamente utilizado en desarrollo web y ciencia de datos. Frameworks: Django (full-stack) y Flask (microframework).
- Java: Ampliamente utilizado en entornos empresariales por su robustez. Framework principal: Spring Boot.
- Node.js / JavaScript: Permite usar JavaScript en el servidor con un modelo de E/S no bloqueante. Frameworks: Express.js, NestJS.
- Ruby on Rails (2004): Popularizó los conceptos de convención sobre configuración y DRY (Don't Repeat Yourself), influyendo a muchos otros frameworks.
- C# / ASP.NET Core: Opción dominante en entornos Microsoft y en el sector financiero y gubernamental.

6.3. Bases de Datos

6.3.1. Bases de Datos Relacionales (SQL)

Los RDBMS organizan los datos en tablas con filas y columnas y utilizan SQL para consultarlos. Se basan en el modelo relacional propuesto por Edgar F. Codd en 1970 [19]. Sus propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad) garantizan la integridad transaccional. Los sistemas más utilizados son MySQL/MariaDB, PostgreSQL, Microsoft SQL Server y SQLite [20].

6.3.2. Bases de Datos NoSQL

Las bases de datos NoSQL surgieron a finales de los años 2000 para dar respuesta a las limitaciones de los RDBMS en escenarios de Big Data y alta disponibilidad [21]. Se clasifican en: documentales (MongoDB, CouchDB), clave-valor (Redis, DynamoDB), columnares (Apache Cassandra) y de grafos (Neo4j). Ofrecen mayor flexibilidad en la estructura de los datos a costa de la consistencia transaccional fuerte [21].

6.4. REST y APIs

REST (Representational State Transfer) es un estilo arquitectónico definido por Roy Fielding en su tesis doctoral del año 2000 [22]. Sus principios fundamentales son: interfaz uniforme mediante recursos identificados por URIs, comunicación sin estado, caché, sistema en capas y separación cliente-servidor. Las APIs RESTful que utilizan JSON como formato de intercambio son el estándar de facto para la comunicación entre frontend y backend. Alternativas modernas incluyen GraphQL (Facebook, 2015) y gRPC (Google), basado en Protocol Buffers y HTTP/2 [22].

7. Seguridad en Aplicaciones Web

7.1. Importancia y OWASP Top 10

La seguridad es una dimensión crítica en el desarrollo de aplicaciones web. La OWASP (Open Web Application Security Project) publica periódicamente su 'OWASP Top 10', la lista de los diez riesgos de seguridad más críticos, ampliamente adoptada por la industria como referencia estándar [23].

7.2. Principales Vulnerabilidades

7.2.1. Inyección SQL

Ocurre cuando el atacante inserta código SQL malicioso a través de campos de entrada que la aplicación no valida adecuadamente. Si la aplicación construye consultas SQL concatenando directamente la entrada del usuario, el código malicioso se ejecuta en la base de datos: robo de datos, modificación de registros, elusión de autenticación [23] [24]. Las medidas de mitigación incluyen: consultas parametrizadas (prepared statements), uso de ORM, validación y escape de entradas, y principio de mínimo privilegio en credenciales de base de datos [24].

7.2.2. Cross-Site Scripting (XSS)

Permite inyectar scripts maliciosos en páginas que serán vistas por otros usuarios. Cuando la víctima carga la página comprometida, el script se ejecuta en su navegador con el contexto de seguridad del sitio legítimo: robo de cookies de sesión, redirección a phishing, registro de pulsaciones de teclado [23]. Las contramedidas incluyen: escapar y codificar salidas hacia el navegador, implementar Content Security Policy (CSP), validar y sanitizar entradas, usar el atributo HttpOnly en cookies.

7.2.3. Autenticación Rota y Gestión de Sesiones

Vulnerabilidades en los procesos de autenticación pueden permitir a los atacantes comprometer contraseñas, tokens o sesiones para asumir la identidad de otros usuarios [23]. Las mejores prácticas incluyen: almacenar contraseñas con funciones de hash seguras (bcrypt, Argon2, PBKDF2, nunca MD5 o SHA-1 sin sal); implementar autenticación multifactor (MFA); gestionar sesiones con tokens aleatorios; invalidar sesiones al cerrar sesión; proteger contra fuerza bruta con bloqueos temporales y limitación de tasa.

7.2.4. Otras Vulnerabilidades OWASP

- Control de acceso roto: Usuarios que realizan acciones fuera de sus permisos modificando parámetros de URL.

- Configuración de seguridad incorrecta: Servidores con configuraciones por defecto inseguras, directorios listados, mensajes de error detallados.
- Componentes con vulnerabilidades conocidas: Bibliotecas o frameworks desactualizados con CVEs conocidos no parcheados.
- CSRF (Cross-Site Request Forgery): El servidor ejecuta peticiones falsificadas enviadas por el navegador de la víctima. Se mitiga con tokens CSRF únicos por sesión.
- Registro y monitorización insuficientes: La falta de logs adecuados dificulta la detección y respuesta ante incidentes.

7.3. Buenas Prácticas de Seguridad (DevSecOps)

- Usar HTTPS en todas las comunicaciones sin excepción [12].
- Implementar cabeceras de seguridad HTTP: Content-Security-Policy, X-Frame-Options, Strict-Transport-Security (HSTS).
- Validar todas las entradas del usuario tanto en el cliente como en el servidor.
- Mantener actualizadas todas las dependencias y aplicar parches de seguridad oportunamente.
- Cifrar datos sensibles en tránsito (TLS) y en reposo (AES-256).
- Realizar pruebas de penetración y revisiones de código periódicas.

8. Arquitecturas Modernas de Aplicaciones Web

8.1. Aplicaciones de Página Única (SPA)

Las Single Page Applications (SPA) cargan una única página HTML y actualizan dinámicamente el contenido sin recargar la página completa, consumiendo APIs REST o GraphQL de manera asíncrona. El resultado es una experiencia fluida similar a las aplicaciones de escritorio nativas [6]. Frameworks como React, Angular y Vue.js están específicamente diseñados para SPAs. Sus limitaciones en SEO y rendimiento inicial impulsaron soluciones de Server-Side Rendering (SSR) como Next.js y Nuxt.js.

8.2. Microservicios

La arquitectura de microservicios estructura una aplicación como un conjunto de servicios pequeños, independientes y desplegados de forma autónoma [25]. Cada servicio implementa una función de negocio específica y se comunica con los demás a través de APIs o mensajería asíncrona (RabbitMQ, Apache Kafka). Ofrece ventajas en escalabilidad, resiliencia y agilidad de desarrollo, aunque introduce complejidad en la comunicación y la observabilidad del sistema [25].

8.3. Computación en la Nube y Serverless

Proveedores como AWS, Google Cloud Platform y Microsoft Azure ofrecen servicios gestionados para construir y escalar aplicaciones globales sin gestionar infraestructura física. El modelo Serverless (FaaS) permite que el desarrollador solo escriba funciones de código que responden a eventos, sin gestionar servidores. El proveedor se encarga del aprovisionamiento, escalado y disponibilidad automáticos (AWS Lambda, Google Cloud Functions, Azure Functions).

9. Rendimiento y Optimización Web

9.1. Web Vitals y Métricas de Rendimiento

El rendimiento tiene impacto directo en la experiencia del usuario y los indicadores de negocio. Google introdujo en 2020 el conjunto de métricas Web Vitals como estándar para medir la experiencia del usuario [26]. Las tres métricas Core Web Vitals son: Largest Contentful Paint (LCP), que mide el tiempo hasta que el elemento de contenido más grande es visible; First Input Delay (FID), que mide la latencia de la primera interacción; y Cumulative Layout Shift (CLS), que mide la

estabilidad visual durante la carga. Estas métricas forman parte de los algoritmos de clasificación de búsqueda de Google.

9.2. Técnicas de Optimización

- Minimización y compresión: Reducir el tamaño de archivos JS, CSS y HTML con herramientas como Webpack o Vite, y comprimir respuestas con Gzip o Brotli.
- Caché: Configurar cabeceras HTTP Cache-Control, ETag y Last-Modified para almacenar recursos estáticos en caché de navegadores y proxies.
- CDN (Content Delivery Network): Distribuir recursos estáticos en servidores geográficamente distribuidos para reducir la latencia.
- Lazy Loading: Cargar imágenes y componentes JavaScript solo cuando son necesarios (al entrar en el viewport).
- HTTP/2 y HTTP/3: Aprovechar la multiplexación de HTTP/2 [10] y la menor latencia de HTTP/3/QUIC [11].

10. Accesibilidad Web y Estándares

10.1. WCAG 2.1

La accesibilidad web implica que los sitios y herramientas web estén diseñados de forma que las personas con discapacidad puedan usarlos. El W3C publica las Pautas de Accesibilidad al Contenido Web [27], el estándar de referencia internacional. Se organiza en cuatro principios: Perceptible, Operable, Comprensible y Robusto (POUR). La accesibilidad beneficia también a usuarios en condiciones adversas (pantallas pequeñas, conexiones lentas) y mejora el SEO.

10.2. El W3C y la Estandarización

El W3C fue fundado en octubre de 1994 por Tim Berners-Lee con la misión de desarrollar estándares que garanticen el crecimiento a largo plazo de la Web [14]. Sus estándares cubren HTML, CSS, XML, SVG, WebRTC y WebAssembly, entre muchos otros. La estandarización garantiza la interoperabilidad entre diferentes navegadores y plataformas, un principio fundamental de la filosofía de la Web abierta.

11. Conclusiones

El estudio de los fundamentos de las aplicaciones web revela la extraordinaria complejidad técnica y la profunda interdependencia entre los componentes que conforman este ecosistema. Desde la propuesta original de Berners-Lee en 1989 [2] hasta las arquitecturas de microservicios [25] y serverless de la actualidad, la Web ha recorrido un camino de transformación constante impulsado por las demandas crecientes de usuarios, desarrolladores y empresas.

La arquitectura cliente-servidor sigue siendo el modelo fundamental sobre el que se construyen todas las aplicaciones web [7] [6], aunque su implementación se ha vuelto exponencialmente más sofisticada con la aparición de las APIs REST [22], los frameworks JavaScript modernos y las plataformas de nube. El protocolo HTTP, diseñado hace más de treinta años, ha demostrado una notable capacidad de evolución hasta su tercera versión principal (HTTP/3, RFC 9114) [11] con mejoras sustanciales de rendimiento.

La seguridad web emerge como una dimensión crítica e ineludible. Las vulnerabilidades documentadas por OWASP [23] [24] demuestran que el desarrollo de aplicaciones web seguras requiere un compromiso constante, educación permanente y la adopción de prácticas de seguridad desde las primeras etapas del desarrollo (DevSecOps). El dominio sólido de los fundamentos presentados en esta investigación es la base imprescindible para navegar con éxito en un paisaje tecnológico en constante cambio.

Bibliografía

- [1] T. C. R. L. A. N. H. F. S. A. Berners-Lee, «The World-Wide Web,» *Communications of the ACM*, p. 76–82, 1994.
- [2] T. Berners-Lee, «Information Management: A Proposal,» 1989. [En línea]. Available: info.cern.ch/Proposal.html.
- [3] B. M. e. a. Leiner, «A Brief History of the Internet,» *ACM SIGCOMM Computer Communication Review*, p. 22–31, 2009.
- [4] T. O'Reilly, «What Is Web 2.0,» 2005. [En línea].
- [5] T. H. J. L. O. Berners-Lee, «The Semantic Web,» *Scientific American*, p. 34–43, 2001.
- [6] R. H. D. E. J. Orfali, *Client/Server Survival Guide*, Nueva York: John Wiley & Sons, 1999.
- [7] A. S. V. S. M. Tanenbaum, *Distributed Systems: Principles and Paradigms*, Upper Saddle River, NJ: Pearson Prentice Hall, 2007.
- [8] R. e. a. Fielding, «Hypertext Transfer Protocol – HTTP/1.1,» 1999.
- [9] R. R. J. Fielding, «HTTP/1.1: Message Syntax and Routing,» 2014.
- [10] M. P. R. T. M. Belshe, «Hypertext Transfer Protocol Version 2 (HTTP/2),» 2015.
- [11] M. Bishop, «HTTP/3,» 2022.
- [12] E. Rescorla, «The Transport Layer Security (TLS) Protocol Version 1.3,» 2018.
- [13] P. Mockapetris, «Domain Names – Concepts and Facilities,» 1987.
- [14] W3C, «HTML5: A Vocabulary and Associated APIs for HTML and XHTML,» 2014.
- [15] M. Pilgrim, *HTML5: Up and Running*, Sebastopol, CA: O'Reilly Media, 2010.
- [16] W. C. W. Group, «CSS Snapshot 2019,» 2019.
- [17] E. International, «ECMAScript 2023 Language Specification (ECMA-262),» 2023.
- [18] D. Flanagan, *JavaScript: The Definitive Guide*, Sebastopol, CA: O'Reilly Media, 2020.
- [19] E. F. Codd, «A Relational Model of Data for Large Shared Data Banks,» *Communications of the ACM*, p. 377–387, 1970.
- [20] R. G. J. Ramakrishnan, *Database Management Systems*, Nueva York: McGraw-Hill, 2003.
- [21] P. J. F. M. Sadalage, *NoSQL Distilled*, Upper Saddle River, NJ: Addison-Wesley, 2012.
- [22] R. T. Fielding, «Architectural Styles and the Design of Network-based Software Architectures,» Irvine, CA, 2000.
- [23] O. Foundation, «OWASP Top 10 – 2021,» 2021.
- [24] O. Foundation, «SQL Injection Prevention Cheat Sheet,» 2021.
- [25] S. Newman, *Building Microservices*, Sebastopol, CA: O'Reilly Media, 2015.
- [26] G. Developers, «Web Vitals,» 2020. [En línea].
- [27] W3C, «Web Content Accessibility Guidelines (WCAG) 2.1,» 2018.

